

The Nostos System



Introducció a les pràctiques de Sistemes Operatius

Un sistema operatiu està format, bàsicament, per 4 mòduls o subsistemes:

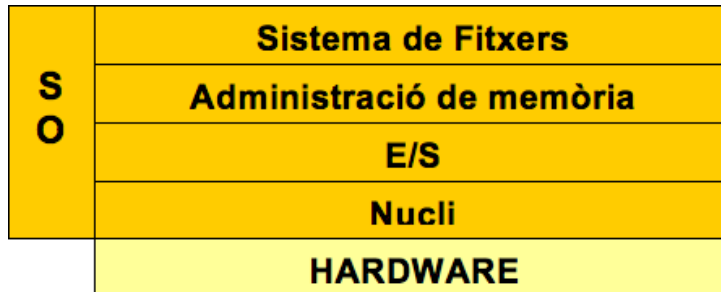


Figura 1. Arquitectura per capes d'un Sistema Operatiu.

NUCLI. És l'única capa que pot inhibir interrupcions. Les seves funcions bàsiques són la representació dels processos en execució, el control de la concurrència de processos, la gestió i control de les interrupcions, i dotar al sistema de mecanismes de comunicació entre processos, de sincronització i exclusió mútua.

SISTEMA E/S. És l'única capa que pot executar instruccions del tipus entrada i sortida. La seva funció és la comunicació amb els perifèrics. Per tant, les capes superiors, per a dialogar amb el *hardware*, ho han de fer a través d'aquesta capa.

GESTIÓ DE MEMÒRIA. Conjuntament amb el *hardware*, crea la possibilitat de disposar de memòria virtual. A més a més, aquesta capa és la responsable de garantir (també conjuntament amb el *hardware*) la protecció de les dades a memòria i també la seva compartició.

SISTEMA DE FITXERS. Dota al sistema d'una visió estructurada de les dades emmagatzemades al disc.

Les aplicacions d'usuari són programes creats per l'usuari. Quan aquests programes necessiten alguna funcionalitat del sistema operatiu, realitzen el que s'anomena una "crida al sistema", que consisteix a fer crides a funcions que ofereixen les diferents capes. És a dir, si l'aplicació vol mostrar un caràcter per pantalla, haurà de cridar a una funció de la capa d'E/S per a aconseguir-ho.

El contingut de la pràctica de l'assignatura de Sistemes Operatius està orientat a l'aprenentatge d'una arquitectura distribuïda mitjançant els diferents mecanismes de comunicació (*sockets* principalment), posant en relleu la problemàtica de la concurrència de processos i la multitasca. L'objectiu principal és establir connectivitat entre els diferents equips i permetre la transmissió d'informació entre els diversos nodes d'un sistema distribuït. A més, cal la utilització de mecanismes del nucli del sistema operatiu (memòria compartida, mètodes d'exclusió mútua, sincronització i creació de processos) per a poder resoldre la pràctica.

NOTA 0. Les quatre pràctiques del curs 2004-2005 rebien els noms de *Arda*, *Arda++*, *Harkonnen* i *IluvatarSon*. Tot era en honor a Frank Herbert i la seva famosa novel·la *Dune*. Força alumnes van recordar l'origen i van descobrir d'on provenia el nom.

NOTA 1. Les quatre pràctiques del curs 2005-2006 rebien els noms de *Oedipus Rex*, *Sphinx*, *Antígona* i *Teiresias*. Com totes les pràctiques de SO, no hi ha res a l'atzar. Els noms provenien de la tragèdia d'Èdip de Sòfocles i dotaven de sentit a les pràctiques. S'iniciava el curs amb una pràctica 1 senzilla, Èdip Rei. Èdip viu feliç com a rei mentre ignora que ha matat el seu pare i s'ha casat amb la seva mare. La ignorància el fa feliç. Els alumnes de SO viuen feliços perquè la pràctica 1 és fàcil i no saben que els espera. La pràctica 2 és l'Sphinx. Èdip ha de superar la prova de l'Sphinx ja que, sinó, aquesta el matarà. Òbviament, la pràctica 2 del curs 2005-2006 era la més difícil de totes i si no la superaves no aprovaves l'assignatura. La pràctica 3 es deia Antígona, qui ajuda a Èdip un cop aquest cau en desgràcia i en la desesperació. Antígona era la salvació dels alumnes que volien anar a l'examen de juny i necessitaven una pràctica llarga lliurada. Us estalvio els detalls de Teiresias. Tot i que molts alumnes van seguir els noms i les referències, cap d'ells va acabar de copsar el sentit de la tragèdia...

NOTA 2. Les pràctiques del curs 2006-2007 es van centrar en l'obra mestra de Dante Alighieri, "La Divina Commedia". Les pràctiques es deien "Inferno", "Purgatorio" i "Paradiso". Suposo que no calen gaires comentaris per entendre què passa a la pràctica 1. La segona pràctica es suavitza, però la necessites si vols presentar-te a examen. Finalment, la tercera és relativament curta i senzilla, un paradís comptant amb tot el que has fet anteriorment.

NOTA 3. Les pràctiques del curs 2007-2008 es van centrar en l'obra mestra dels germans Wachowski, "Matrix", on els humans (alumnes) han de lluitar contra les màquines per a sobreviure (aprovar l'assignatura). Les pràctiques es deien "Matrix", "El Ferroviari" i "Keymaker". Matrix era el servidor central on s'havien de connectar els clients, com *Nebuchadnezzar*, per a poder intercanviar arxius i comunicar-se mitjançant un xat distribuït. També podien connectar-se amb altres dimonis, com *Oraculo* o *Link*, que els donaven consells. *El Ferroviari* era una pràctica de simulació de càrrega de processos i administració de memòria. Si coneixeu el paper de *El Ferroviari* a la pel·lícula *Matrix* veureu que la relació és directa. Finalment, la codificació de fitxers en EXT2. Vam considerar que un bon nom per descodificar era el personatge de *Matrix* anomenat *Keymaker*.

NOTA 4. Les pràctiques del curs 2008-2009 es van centrar en l'obra mestra d'Isaac Asimov, "La fundació". Les pràctiques es deien: "Trantor", el planeta central de la galàxia, tenint, per tant, la mateixa magnitud que la pràctica 1 de Sistemes Operatius; "Trevize, the loader", nom del conseller de la primera fundació, amb una intuïció insòlita, però amb perilloses intencions; i "Pelorat, the file reader", nom del professor d'història que ajudà a Trevize a trobar el planeta Terra, la llar on descansar després d'aprovar les tres pràctiques de Sistemes Operatius.

NOTA 5. Les pràctiques del curs 2009-2010 es van centrar en el grup australià de *hard rock* AC/DC, com a homenatge a la marxa del que va ser professor de pràctiques de l'assignatura els tres darrers anys, Hugo Meza. La primera pràctica, sota el nom de *Highway to shell* (petita modificació del nom de la cançó més popular del grup), conduïa als alumnes per tres fases. La primera, *Welcome to the Shell*, donava la benvinguda a l'infern als alumnes amb la implementació d'un intèrpret de comandes *shell*, sota el nom de Malcolm (guitarrista del grup). A la segona fase, per tal de fugir de les tenebres en què es trobaven, els alumnes es van veure pactant amb el diable (*Dealing with the Devil*), on havien de seguir tot un protocol per a

comunicar-se amb el dimoni Angus (guitarrista del grup). Finalment, quan creien que tot havia passat, es topaven amb *Ballbreaker* (disc que van gravar el 1995), nom que no requereix gaires explicacions. La segona pràctica del curs 2009-2010 rebia el nom de *Back in Black*, nom del disc llençat el 1980, on els alumnes havien d'identificar el format d'un volum donat i extreure'n informació.

NOTA 6. Les pràctiques del curs 2010-11 es van centrar en les obres del polèmic *Donatien Alphonse François de Sade*, més conegut com "el Marquès de Sade". Així, la pràctica es titulava "*Les 120 journées de Sodome*". Aquesta estava dividida en tres fases: "*Le Château de Silling*" era la primera d'elles, on els alumnes gestionaven el comportament del client de l'aplicatiu, el qual rebia el nom del personatge de la novel·la "Blangis". Posteriorment, els alumnes s'endinsaven al castell en una segona fase anomenada "*Les quatre madames*", on Blangis es connectava a un dimoni anomenat "Thérèse". Finalment, la cosa es desmadrava a la tercera fase, on els alumnes havien de crear el servidor "*Libertinage*", que, com el seu nom indica, és on començava el "festival".

NOTA 7. Les pràctiques del curs 2011-12 es van centrar en la mundialment coneguda sèrie televisiva del Simp(so)ns. Aquesta pràctica estava dividida en cinc fases: "La shell de Homer", "el bar de Mou", "Clancy Wiggum", "Living in Springfield" i "Living in Springfield++", en conjunt aquestes fases formaven un sistema que permetia executar comandes de forma local, algunes pròpies en un servidor remot i l'activació de diversos serveis que mostraven frases mítiques de la sèrie, tot això seguint una arquitectura client - servidor.

NOTA 8. La pràctica del curs 2012-13 es va anomenar LsBox. El motiu va ser força simple: es tractava de dissenyar i implementar un sistema molt similar (de fet simplificat) del conegut Dropbox. L'únic detall curiós era el logotip de la pràctica que portava un mapa d'Austràlia encobert doncs era un petit homenatge a un monitor de SO que deixava aquestes tasques.

NOTA 9. La pràctica del curs 2013-14 es va anomenar LsHangIn. El motiu era que era una versió simplificada del conegut Google Hangouts: es tractava de dissenyar i implementar un sistema de xats amb múltiples sales per a usuaris. A més, cadascuna de les Fases tenien relació amb la pel·lícula Ressaca a Las Vegas.

NOTA 10. La pràctica del curs 2014-15 es va anomenar Gekko. Gekko és un empresari i principal protagonista de la pel·lícula Wall Street, relació directa amb la Borsa i l'objectiu de la pràctica. Però, a més, hi havia diversos homenatges més: TumblingDice, el generador de fluctuacions, és també una cançó dels Rolling Stones i Dozer, l'operador de Borsa, és un dels personatges de Matrix.

NOTA 11. La pràctica del curs 2015-16 es va anomenar LsTransfer, the force awakens. Era un clar homenatge al retorn de la saga d'Star Wars. A més el servidor es deia Naboo (planeta que surt en diferents episodis d'Star Wars) i els clients Gungan (habitants d'aquest planeta).

NOTA 12. La pràctica del curs 2016-17 es va anomenar LsTinder, may the Love be with you. Era per la clara similitud a la xarxa social a la qual la pràctica feia referència i el lema un altre homenatge a la saga Star Wars. A més, els diferents processos es deien Rick i Morty en referència a una sèrie americana de televisió d'animació per a adults.

NOTA 13. La pràctica del curs 2017-18 es va anomenar LsEat, may the Food be with you. Primerament, el nom en referència a la moda del Just Eat. El lema un altre homenatge a la saga Star Wars, en la seva imminent estrena de l'Episodi VIII, The Last Jedi. A més, els diferents processos tenien noms d'homenatge a StarTrek. Picard i Data personatges i Enterprise, la nau.

NOTA 14. La pràctica del curs 2018-19 es va anomenar Cosgrove System, Stairway to heaven. Primerament, Cosgrove és el cognom de la família protagonista d'una pel·lícula mítica de Peter Jackson: Braindead, considerada un clàssic de les pel·lícules gore de l'època. El lema, Stairway to Heaven, no era res relacionat amb els observatoris de la pràctica sinó un homenatge a la famosa cançó de Led Zeppelin. A més, els diferents processos tenien noms dels protagonistes de la pel·lícula Braindead: McGruder, McTavish, Paquita i Lionel.

NOTA 15. La pràctica del curs 2019-20 es va anomenar Cypher System era un homenatge de nou a Matrix, perquè s'estava gravant pel 2022 Matrix IV, un mite en el món de la ciència-ficció. L'script de la portada si l'executàveu feia les lletres verdes caient per la pantalla de Matrix. Trinity és la protagonista principal de Matrix i els exemples sempre són noms de personatges de la saga. No hi ha res a l'atzar.

NOTA 16. La pràctica del curs 2020-21 es va anomenar Overlook System. Això era un homenatge a la mítica pel·lícula de El Resplandor (the Shining) d'Stanley Kubrick on el 2020 feia 40 anys de la seva estrena. L'Hotel s'anomenava Overlook i la imatge de la portada de la pràctica era la d'un dels seus mítics passadissos. Els diferents processos del sistema a dissenyar eren Jack, Wendy, Danny i Lloyd que són noms de personatges de la saga. No hi ha res a l'atzar.

NOTA 17. La pràctica del curs 2021-22 es va anomenar Arrakis System. Això era un homenatge a la mítica pel·lícula de Dune de Denis Villeneuve del 2021, però pensant amb l'original de David Lynch de 1984. Arrakis és el planeta on es desenvolupa la gran part de la pel·lícula. Hi havia els Atreides (els bons), els Harkonnen (els dolents) i els Fremen (la raça nativa del planeta Arrakis) que són elements importants i claus a la mítica pel·lícula.

NOTA 18. La pràctica del curs 2022-23 es va anomenar Eä System. Això era un homenatge a la sèrie El Senyor dels Anells: Els anells de poder (originalment en anglès, The Lord of the Rings: The Rings of Power). És una sèrie de televisió basada en la novel·la El Senyor dels Anells i els seus apèndixs de J. R. R. Tolkien. Transcorre milers d'anys abans d'El hòbbit i El Senyor dels Anells de Tolkien a la Segona Era de la Terra Mitjana. Està produïda per Amazon Studios.

NOTA 19. La pràctica del curs 2023-24 es va anomenar HAL 9000 System. Això era un homenatge HAL 9000, personatge mític de la coneguda sèrie de pel·lícules Odissea espacial; en concret 2001: una odissea de l'espai. 2001: una odissea de l'espai (títol original en anglès: 2001: A Space Odyssey) és una pel·lícula de ciència-ficció del 1968, dirigida per Stanley Kubrick, escrita per ell mateix i per Arthur C. Clarke. La pel·lícula tracta temes com l'evolució humana, la intel·ligència artificial, el futur i la vida extraterrestre. David Bowman i Frank Poole eren dos dels protagonistes, i per això els noms dels processos. La nau on viatjaven es deia Discovery (un altre procés) i el Monolit era un altre element important a la pel·lícula.

NOTA 20. La pràctica del curs 2024-25 es va centrar en la figura d'Arthur Fleck, el Joker. El projecte es deia Mr. J. System, en honor a com l'anomena la Harley. El sistema tenia un servidor central, Gotham, i clients Fleck, que feien peticions als processos Enigma i Harley, sota la vigilància d'Arkham, element clau en l'univers de Batman.

NOTA 21. La pràctica del curs 2025-26 es va anomenar The Citadel System i estava ambientada en l'univers de Joc de Trons. La Ciutadella és l'organització on es formen els Mestres dels Set Regnes, i per això els processos principals rebien el nom de Maester. Cada Maester representava una de les Grans Cases de Westeros i havia de comunicar-se amb la resta de regnes per forjar aliances i comerciar. Com que els missatges podien haver de travessar diversos regnes intermedis, la xarxa reproduïa les rutes dels corbs i dels emissaris pel continent. Els processos auxiliars s'anomenaven Envoy i permetien gestionar diverses missions simultàniament. Els segells de les cases, els corbs, els llibres de comptes i els noms dels diferents regnes completaven les referències a aquest univers. Com sempre, no hi havia res a l'atzar.

Ara us toca a vosaltres identificar la relació de la pràctica d'aquest any 2026-27 i posar-ho a la memòria.

The Nostos System

La guerra de Troia ha acabat, però el retorn a casa no serà fàcil. El mar Egeu s'ha convertit en un territori perillós: els ports estan saturats, les illes protegeixen els seus recursos i els vents de Posidó amenacen qualsevol vaixell que s'aventuri massa lluny. A més, les provisions s'esgoten ràpidament quan els navegants han d'esperar davant d'un port sense poder atracar.

Enmig d'aquest caos, Ithaca necessita restablir les comunicacions amb la resta de l'arxipèlag. Diverses illes esperen objectes, relíquies i missatges que només poden ser transportats per mar. Per aconseguir que aquests encàrrecs arribin al seu destí, caldrà dissenyar **The Nostos System**, una xarxa distribuïda de missions de transport marítim. El procés Ithaca gestionarà els viatges disponibles. Els diferents processos Odysseus s'hi connectaran per consultar els contractes i acceptar-ne un. Cada viatge només podrà ser realitzat per un vaixell alhora, tot i que diversos Odysseus podran navegar simultàniament en trajectes diferents. En acceptar una missió, el vaixell rebrà l'objecte que haurà de transportar i la localització de la primera illa.

Durant el recorregut, els Odysseus es connectaran directament als processos Island. Cada illa tindrà un port de capacitat limitada i, si està complet, els vaixells hauran d'esperar consumint les seves provisions. Un cop dins del port, podran comerciar, comprar menjar i obtenir el mapa que els permetrà continuar cap a la següent destinació.

Quan un Odysseus arribi a l'illa final, haurà d'entregar l'objecte transportat. Si l'entrega és correcta, l'illa li proporcionarà un comprovant segellat que haurà de presentar a Ithaca per completar el contracte i cobrar la recompensa. Si el vaixell es queda sense provisions o no pot continuar, es considerarà perdut al mar i la missió fracassarà.

La vostra tasca serà dissenyar i implementar tot el programari necessari per gestionar aquest ecosistema, prestant especial atenció a la concurrència, la sincronització, la transferència fiable de fitxers i la persistència de les dades. Per facilitar el desenvolupament, la pràctica es dividirà en tres fases incrementals.

Descripció General

Una primera aproximació funcional de l'arquitectura de processos del sistema és la que es presenta a la Figura 2. L'arquitectura estarà formada per tres tipus de processos independents: **Ithaca**, que actuarà com a servidor central del sistema; múltiples processos **Odysseus**, que representaran els vaixells encarregats de completar els diferents viatges; i diversos processos **Island**, corresponents a les illes que formen les diferents rutes marítimes.

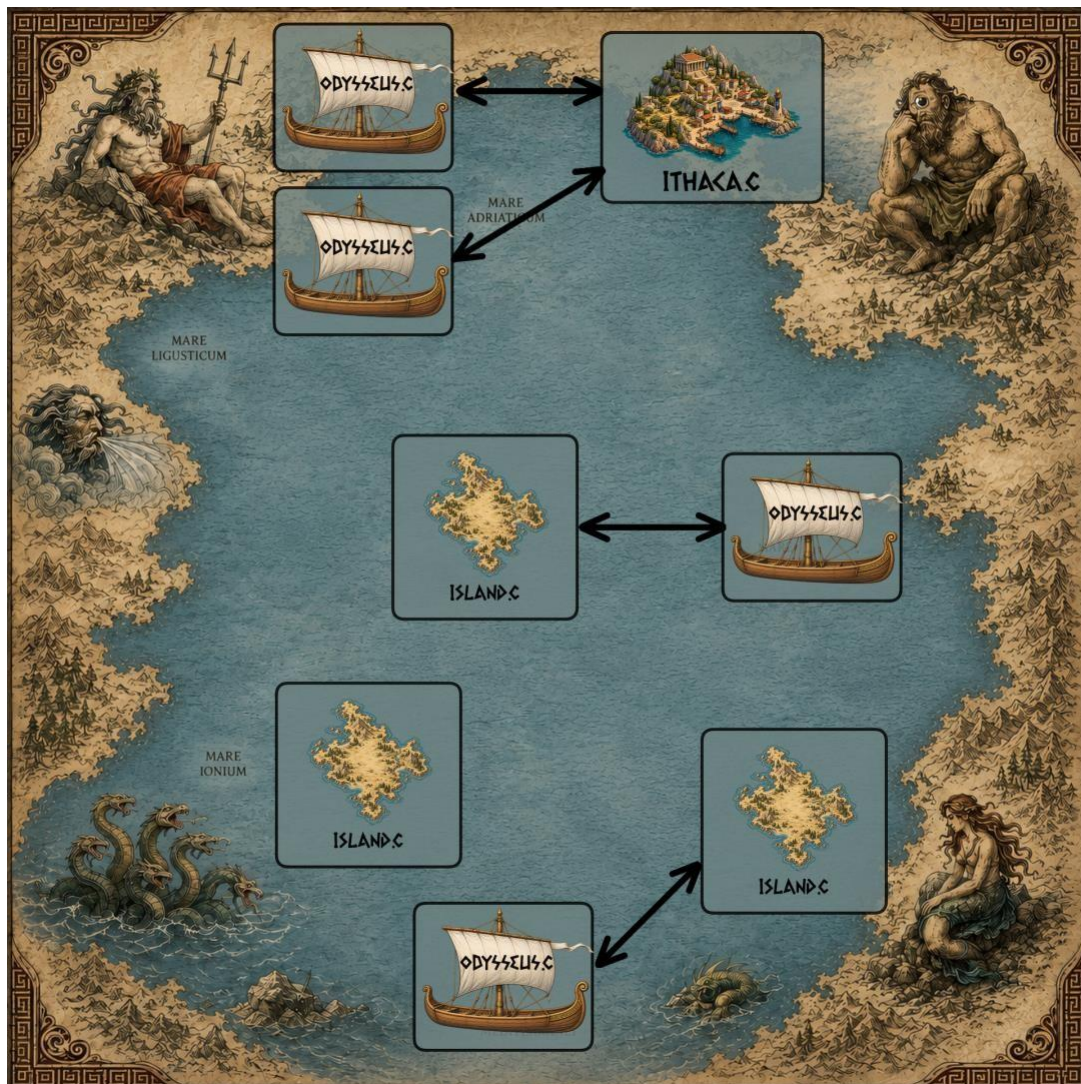


Figura 2. The Nostos System: esquema funcional.

El procés **Ithaca** serà l'encarregat de gestionar els viatges disponibles i coordinar la sortida dels diferents vaixells. Els processos **Odysseus** s'hi connectaran per consultar els trajectes que encara es poden realitzar i acceptar-ne un. Cada viatge només podrà estar assignat a un **Odysseus** alhora, tot i que diversos vaixells podran estar navegant simultàniament per rutes diferents.

Un cop acceptat un viatge, l'**Odysseus** abandonarà **Ithaca** i es farà a la mar, iniciant el recorregut per les diferents illes que formen la seva ruta. Per fer-ho, s'anirà connectant successivament als processos **Island** corresponents. Cada illa disposarà d'un port amb una capacitat limitada de vaixells i, si en arribar totes les places estan ocupades, l'**Odysseus** haurà d'esperar davant de la costa fins que se li permeti atracar. Durant aquesta espera continuarà consumint provisions i, si aquestes s'esgoten abans de poder entrar al port, el vaixell es considerarà perdut al mar.

Quan un **Odysseus** aconsegueixi finalment atracar, podrà interactuar amb l'illa. Entre altres accions, podrà comerciar amb els seus habitants, comprar les provisions necessàries per continuar navegant i demanar el mapa que li indicarà com arribar fins a la següent illa del trajecte. D'aquesta manera, cada **Odysseus** avançarà de port en port fins arribar a la destinació final de la seva missió.

A l'última illa del trajecte, l'**Odysseus** haurà d'entregar l'objecte que ha transportat durant el viatge. Si l'entrega és correcta, l'illa li proporcionarà un comprovant segellat que certificarà que l'encàrrec ha arribat al seu destí. Amb la missió completada, l'**Odysseus** podrà emprendre el viatge de tornada cap a **Ithaca**, on es connectarà novament al servidor per presentar el comprovant i reclamar la recompensa corresponent abans de fer-se a la mar en una nova aventura.

El sistema haurà de permetre que múltiples **Odysseus** naveguin simultàniament, coordinant tant l'assignació exclusiva dels viatges a **Ithaca** com l'accés concurrent als ports de capacitat limitada de les diferents illes. Al llarg de la pràctica també s'introduiran mecanismes de transferència de fitxers i de persistència, de manera que determinades dades de l'**Odysseus** es puguin conservar entre execucions.

Per poder dissenyar i implementar correctament la pràctica, es recomana fer una **lectura completa de l'enunciat** abans de començar-ne el desenvolupament.

Per facilitar-ne la realització, la pràctica s'ha dividit en **3 fases incrementals**:

1. **A la fase 1** es desenvoluparà l'estructura bàsica dels processos **Odysseus**, **Ithaca** i **Island**. S'implementarà la lectura dels seus respectius fitxers de configuració i el terminal de comandes d'**Odysseus**, des d'on l'usuari podrà consultar la informació necessària i donar les ordres que guiaran el seu viatge.
2. **A la fase 2** s'implementarà la comunicació entre els diferents processos. Els **Odysseus** podran arribar simultàniament a **Ithaca** per consultar i acceptar viatges i, un cop assignada una ruta, salpar cap a les diferents illes que la formen. Els processos **Island** hauran de gestionar la capacitat limitada dels seus ports i l'espera dels vaixells que no puguin atracar immediatament. Un cop a port, els

Odyseus podran comerciar, obtenir provisions i aconseguir el mapa necessari per continuar navegant.

3. **A la fase 3** es completarà el cicle de les missions incorporant la transferència de l'objecte transportat i del comprovant generat a l'illa de destinació. Després de completar l'entrega, l'Odyseus podrà tornar a **Ithaca** i presentar-hi aquest comprovant perquè el servidor validi la missió i entregui la recompensa corresponent. Finalment, s'incorporarà la persistència de l'estat d'Odyseus, de manera que informació com el seu inventari o el saldo acumulat es pugui conservar entre execucions.

Fase 1: Initaca

En aquesta primera fase es construiran les bases de **The Nostos System**. Abans que cap Odysseus pugui abandonar Ithaca i aventurar-se pel mar Egeu, els diferents processos hauran de conèixer el seu entorn, carregar correctament la informació que necessitaran durant el viatge i estar preparats per rebre les ordres de l'usuari.

Es començaran a dissenyar i programar els tres processos principals del sistema: **Odysseus.c**, **Ithaca.c** i **Island.c**. Tots tres disposaran del seu propi fitxer de configuració, que determinarà com s'ha de comportar cada instància. Així, un mateix executable island permetrà representar qualsevol de les illes de l'arxipèlag simplement iniciant-lo amb una configuració diferent.

En aquesta fase **encara no s'implementarà cap comunicació de xarxa ni cap mecanisme de concurrència**. L'objectiu serà llegir i emmagatzemar correctament els diferents fitxers d'inicialització i construir el terminal de comandes d'Odysseus. Les comandes s'hauran de reconèixer i validar sintàcticament, però la seva funcionalitat s'anirà incorporant en les fases següents.

Fitxer de configuració – Odysseus (odysseus.dat)

Cada **Odysseus** disposarà del seu propi fitxer de configuració. Aquest contindrà la informació necessària per identificar el vaixell, localitzar Ithaca, conèixer el punt d'inici de qualsevol travessia i establir els recursos amb què començarà el seu viatge.

El fitxer tindrà, en aquest ordre:

- La ruta de la carpeta on guardarà els seus arxius.
- La IP del procés Ithaca i port
- La informació de l'illa inicial de qualsevol travessia, en format <IP> <PORT>. Aquesta illa serà sempre **Aeaea**.
- La quantitat inicial de diners.
- El nombre inicial d'aliments diferents.
- Llista d'aliments inicial de la tripulació i la seva quantitat, expressada en quilograms, en format <PRODUCT> <AMOUNT>.

Per exemple:

```
./polyphemus_files
Polyphemus 172.16.214.18 9001
172.16.214.20 9002
250
2
Barley 40
DriedFigs 60
```

Figura 3. Fitxer de configuració d'un procés Odysseus.

En fases posteriors, aquesta informació permetrà a l'Odysseus arribar fins a Ithaca, acceptar un encàrrec i preparar-se per a la travessia. En aquesta primera fase, el procés només haurà de llegir el fitxer i emmagatzemar-ne correctament la informació en les estructures de dades corresponents.

Fitxer de configuració – Ithaca (ithaca.dat)

Ithaca serà el punt de trobada dels diferents Odysseus i l'encarregada de gestionar els encàrrecs disponibles i custodiar els objectes que s'hauran de transportar. El seu fitxer de configuració contindrà:

- El nom del servidor.
- La ruta de la carpeta on es troben els arxius dels encàrrecs.
- La IP on Ithaca escoltarà connexions en fases posteriors i el port associat.

Per exemple:

```
Ithaca
./objects
172.16.214.18 9001
```

Figura 4. Fitxer de configuració d'un procés Ithaca.

Fitxer de viatges – Ithaca (voyages.dat)

Ithaca conservarà en un segon fitxer el registre dels encàrrecs disponibles. Cada viatge identificarà l'objecte que s'ha de transportar, el fitxer que el conté, l'illa on s'haurà d'entregar i la recompensa que rebrà l'Odysseus si aconsegueix completar la missió. Aquest fitxer es proporcionarà a l'eStudy i no inclourà cap identificador. Ithaca haurà de generar i gestionar internament els identificadors dels viatges.

Cada viatge ocuparà una línia amb el format:

<OBJECT> <FILE> <DESTINATION> <REWARD>

Per exemple:

```
OdysseusBow ./obj/odysseus_bow.png Ismarus 1050
TridentOfPoseidon ./obj/trident_of_poseidon.png Scheria 1500
CircesWand ./obj/circes_wand.png Ogygia 1000
```

Figura 5. Exemple del fitxer voyages.dat d'Ithaca.

Ithaca **no coneixerà la ruta necessària per arribar fins a la destinació ni la localització dels diferents ports**. Un cop l'Odysseus accepti un encàrrec, haurà de començar la

travessia des d'Aeaea i anar descobrint el camí fins a la seva destinació a mesura que visiti noves illes.

Fitxer de configuració – Island (island.dat)

Cada procés **Island** representarà una illa diferent del mar Egeu. En executar el programa amb una configuració determinada, el procés adoptarà la identitat, el port, el mercat i les connexions marítimes de l'illa corresponent. D'aquesta manera, un mateix executable island permetrà aixecar totes les illes necessàries simplement utilitzant fitxers de configuració diferents.

El fitxer de configuració contindrà:

- El nom de l'illa.
- La ruta de la carpeta on emmagatzemarà els seus arxius.
- La IP i port de la illa on escoltarà connexions en fases posteriors.
- La capacitat màxima del port.
- Una secció --- ROUTES --- amb les illes a les quals es pot navegar directament.

Cada ruta ocuparà una única línia i seguirà sempre el format:

<NAME> <IP> <PORT>

on NAME identifica l'illa veïna i IP i PORT indiquen on es troba el seu procés Island. Totes les rutes seran **bidireccionals**: una connexió entre dues illes es podrà recórrer en ambdós sentits.

Per exemple:

```
Aeaea
./aeaea_files
172.16.214.20 9002
2
--- ROUTES ---
Aeolia 172.16.214.18 9003
Ismarus 172.16.214.21 9004
Scheria 172.16.214.20 9005
```

Figura 6. Fitxer de configuració del procés Island corresponent a Aeaea.

Cada illa **només coneixerà les seves connexions directes**. Així, Aeaea podrà saber com arribar fins a Aeolia, Ismarus o Scheria, però no coneixerà quines rutes existeixen des d'aquestes illes.

Les rutes presents al fitxer de configuració **no es podran considerar directament vàlides**. Un cop carregada la configuració d'una illa, les seves connexions s'hauran de **validar obligatòriament mitjançant la funció *SPHRAGIS_filter_island_configuration()*** proporcionada per la llibreria **SPHRAGIS**.

Aquesta funció eliminarà de la configuració totes aquelles destinacions que no estiguin realment connectades amb l'illa indicada. A partir d'aquest moment, **únicament les connexions que hagin superat aquest filtratge es consideraran rutes directes vàlides** i podran ser comunicades posteriorment als Odysseus.

Per tant, encara que una ruta aparegui escrita al fitxer island.dat, no implica necessàriament que sigui una connexió navegable.

A mesura que un Odysseus arribi a nous ports i compri els mapes locals, incorporarà aquestes connexions bidireccionals al seu mapa. Serà, per tant, el mateix navegant (vosaltres) qui haurà d'anar descobrint el camí que el portarà fins a la destinació de la seva missió. Per incorporar les connexions al seu mapa, es farà ús de la llibreria SPHRAGIS proporcionada a l'eStudy. Consulteu-ne la documentació per veure com incorporar-la al projecte i com proporcionar-li les rutes conegudes perquè dibuixi el mapa al terminal.

Fitxer de stock – Island (stock.db)

Cada illa disposarà del seu propi mercat i, per tant, d'un conjunt diferent de productes, quantitats i preus. Aquesta informació es guardarà en un fitxer binari associat a cada procés Island. Les **provisions** comprades a les illes permetran als Odysseus reposar el menjar consumit durant les esperes davant dels ports.

Cada entrada del fitxer representarà un producte i contindrà els camps següents:

- **Nom:** char name[100]
- **Quantitat:** int amount, en **quilograms**
Preu: int price, expressat en **gold per quilogram**

Els registres s'emmagatzemaran consecutivament al fitxer i es podran llegir fins arribar al final d'aquest.

En aquesta primera fase, els processos Island només hauran de **llegir el fitxer binari i carregar-ne el contingut a memòria**. La compra, la venda i l'actualització del stock s'introduiran en fases posteriors. Els fitxers stock.db de les illes es proporcionaran a través d'eStudy.

Terminal de Comandes

El procés **Odysseus** serà l'únic procés del sistema que disposarà d'un terminal **interactiu**. Des d'aquest terminal, el navegant podrà donar les ordres que, a mesura que avanci la pràctica, li permetran acceptar encàrrecs, fer-se a la mar, comerciar i explorar les diferents illes.

En aquesta primera fase, però, **no caldrà implementar la funcionalitat de les comandes**. L'objectiu serà només construir un parser capaç de reconèixer-les correctament, validar-ne la sintaxi i extreure els seus arguments.

Les paraules que formen les comandes seran *case insensitive*.

COMANDA	DESCRIPCIÓ
CONNECT ITHACA	Connectarà l'Odysseus amb Ithaca (si no hi ha cap altra connexió amb Ithaca activa).
LIST VOYAGES	Consultarà els encàrrecs disponibles, només connectat a Ithaca i sense cap viatge actiu.
ACCEPT <ID>	Acceptarà un viatge de l'últim LIST VOYAGES, només connectat a Ithaca i sense cap viatge actiu.
SAIL <ISLAND>	Navegarà des d'Ithaca o del port actual fins a la illa indicada, sempre i quan aquesta sigui coneguda, o un viatge vàlid des de la ubicació actual, tingui viatge actiu o sense.
MAP	Mostrarà localment amb Sphragis el mapa i les rutes descobertes. Disponible en qualsevol moment.
LIST MARKET	Consultarà el mercat de l'illa, només quan l'Odysseus hi estigui atracat.
BUY <PRODUCT> <AMOUNT>	Comprarà una quantitat positiva estant atracat. Es podrà repetir BUY MAP 1; cada compra costarà 50 gold.
SELL <PRODUCT> <AMOUNT>	Vendrà una quantitat positiva d'un producte transportat, només estant atracat.
STATUS	Mostrarà localment diners, càrrega i viatge actiu, si n'hi ha. Disponible en qualsevol moment.
DELIVER	Entregarà l'objecte del viatge actiu, només atracat a l'illa de destinació.

CLAIM	Reclamarà a Ithaca la recompensa d'una missió completada, només connectat després de l'entrega.
-------	---

Taula 1. Llistat de comandes esteses.

Per a aquesta fase, qualsevol ordre reconeguda i amb una sintaxi correcta mostrarà simplement:

Command OK

No caldrà comprovar encara si l'ordre té sentit segons l'estat actual del viatge. Per exemple:

\$ BUY DriedFish 3
Command OK

serà una ordre correcta encara que l'Odysseus no estigui atracat en cap port.

En canvi, si l'usuari introdueix una ordre que no existeix, el programa mostrarà:

Unknown command

Si l'ordre existeix però els seus arguments no són correctes, el programa haurà d'indicar la sintaxi esperada.

Quan una comanda requereixi valors numèrics, com l'identificador d'un viatge o la quantitat d'un producte, també s'haurà de comprovar que aquests arguments siguin numèricament vàlids.

Execució dels processos

Els tres processos s'executaran indicant per paràmetre el fitxer de configuració que han d'utilitzar:

- ./odysseus <config.dat>
- ./ithaca <config.dat> <voyages.dat>
- ./island <config.dat> <stock.db>

Ithaca i **Island** no disposaran de terminal de comandes. Després de carregar correctament la seva configuració i els fitxers associats, mostraran un missatge d'inicialització i romandran actius fins que l'usuari en finalitzi l'execució mitjançant CTRL+C. **Només pot executar-se un procés Ithaca alhora.**

Per exemple:

```
user.name@montserrat:~> ./ithaca ithaca.dat voyages.dat
Ithaca initialized. 24 voyages loaded.
Waiting for Odysseus...
^C
Ithaca closes the harbor.
```

Figura 7. Execució d'Ithaca.

```
user.name@montserrat:~> ./island aeaea.dat stock.db
Island Aeaea initialized.
Port capacity: 1 ship.
2 sea routes loaded.
8 products available.
^C
Aeaea closes its port.
```

Figura 8. Execució d'Island.

El procés Odysseus, en canvi, mostrarà el terminal interactiu:

```
user.name@montserrat:~> ./odysseus odysseus.dat
Odysseus Polyphemus is ready to sail.

$ CONNECT ITHACA
Command OK

$ LIST VOYAGES
Command OK

$ ACCEPT 2
Command OK

$ SAIL Aeaea
Command OK

$ MAP
Command OK

$ BUY DriedFish 3
Command OK

$ ACCEPT
Usage: ACCEPT <voyage_id>

$ something else
Unknown command
```

Figura 9. Execució d'Odysseus.

Es demana

En aquesta primera fase es demana:

- Dissenyar i programar els executables Odysseus, Ithaca i Island.

- Permetre que cadascun dels processos rebi per paràmetre el seu fitxer de configuració.
- Validar obligatòriament les rutes carregades de cada procés Island mitjançant `SPHRAGIS_filter_island_configuration()` abans d'utilitzar-les o emmagatzemar-les com a connexions vàlides.
- Llegir i emmagatzemar correctament la configuració del procés Odysseus.
- Llegir la configuració del procés Ithaca i carregar correctament el seu fitxer `voyages.dat`.
- Llegir la configuració dels processos Island, incloent-ne les rutes marítimes.
- Llegir i carregar a memòria els fitxers binaris `stock.db` de les illes.
- Emmagatzemar tota la informació llegida en estructures de dades adequades.
- Implementar el terminal interactiu d'Odysseus.
- Reconèixer totes les ordres de la Taula 1 i validar-ne la sintaxi i els arguments.
- Mostrar Command OK quan una comanda sigui sintàcticament correcta, sense implementar-ne encara la funcionalitat.
- Detectar correctament les comandes desconegudes i les comandes conegudes amb una sintaxi incorrecta.
- Permetre finalitzar Odysseus mitjançant CTRL+C.
- Permetre finalitzar Ithaca i qualsevol procés Island mitjançant CTRL+C.
- Garantir que qualsevol procés alliberi correctament els recursos utilitzats abans de finalitzar.

En aquesta fase **no s'ha d'implementar cap socket, connexió de xarxa ni mecanisme de concurrència**.

Consideracions Fase 1

- Un cop finalitzada l'execució de qualsevol procés s'haurà **d'alliberar** tota la memòria dinàmica significativa, tancar els fitxers que continuïn oberts i deixar correctament alliberats els recursos utilitzats.
- Des d'aquesta primera fase i fins al final de la pràctica, tots els processos hauran de poder finalitzar correctament en rebre CTRL+C.
- Les comandes introduïdes al terminal són *case insensitive*.
- Podem considerar que el **format dels fitxers de configuració proporcionats o creats segons les especificacions és correcte**.
- **No** es poden utilitzar les funcions `printf`, `scanf`, `gets`, `puts`, etc. Només es podrà interactuar amb la pantalla i amb els fitxers mitjançant les funcions `read` i `write`. Sí que es permet l'ús de `asprintf` i funcions similars.

- **No** es poden utilitzar les funcions system, popen, stat així com cap de les seves variants.
- Cal garantir l'estabilitat de l'aplicació i el seu correcte funcionament. En cap cas es poden produir bucles infinits, core dumps, esperes actives, accessos incorrectes a memòria o warnings durant la compilació.
- Tots els possibles errors d'obertura de fitxers, lectura, escriptura o reserva de memòria s'hauran de gestionar adequadament. En cas que es produeixi un error, se n'haurà **d'informar l'usuari** i, sempre que sigui possible, continuar o finalitzar de manera controlada.
- És obligatori utilitzar un **Makefile** per generar els executables.
- **El codi haurà d'estar degudament modulat.** No es podrà concentrar tota la implementació en un únic fitxer .c.
- La no entrega de qualsevol dels mòduls o executables obligatoris demanats en aquesta fase implicarà que la Fase 1 es consideri suspesa.
- No és suficient que l'arxiu que pugeu al pou tingui extensió .tar, sinó que s'ha de poder desempaquetar amb la comanda tar. Qualsevol pràctica o checkpoint que no es pugui "descomprimir" d'aquesta manera **no serà corregida**.
- Tot i que no serà obligatori entregar un document de disseny abans de començar aquesta fase, es recomana fortament definir prèviament els mòduls del sistema, les estructures de dades necessàries i les responsabilitats de cada procés, i validar aquest disseny amb els monitors de l'assignatura abans de començar a programar. **Reviseu l'Annex III per a més detalls.**

Fase 2: Ports de l'Egeu

En aquesta segona fase, els diferents processos de **The Nostos System** començaran finalment a comunicar-se. Els Odysseus podran arribar fins a **Ithaca** per consultar els encàrrecs disponibles i acceptar-ne un i, un cop preparats, abandonar el port i començar la seva travessia pel mar Egeu.

A partir d'aquest moment, els processos **hauran de poder executar-se i comunicar-se entre màquines diferents**. Tota la comunicació entre Odysseus, Ithaca i les diferents Island s'haurà d'implementar mitjançant sockets, seguint el protocol de comunicació definit a l'Annex II.

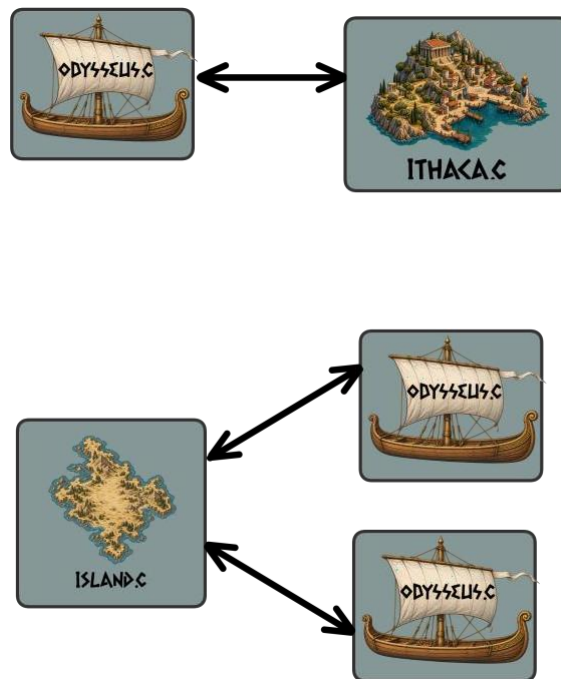


Figura 10. Esquema funcional de la Fase 2.

Durant l'execució del sistema, **no es permetrà assumir que tots els processos es troben a la mateixa màquina**. Com a mínim, els processos **Ithaca i Aeaea s'hauran d'executar obligatòriament en màquines diferents**. La resta de processos **Island** s'hauran de distribuir entre les diferents màquines disponibles; no és necessari disposar d'una màquina exclusiva per a cada illa.

En conseqüència, les connexions s'hauran d'establir sempre utilitzant les IP i els ports indicats als fitxers de configuració, sense assumir adreces locals ni compartir informació entre processos per mecanismes que només funcionin dins d'una mateixa màquina.

El sistema haurà de permetre que múltiples Odysseus naveguin simultàniament. Això implicarà coordinar l'accés als viatges disponibles a Ithaca, gestionar ports amb capacitat limitada i permetre que diversos navegants interactuïn al mateix temps amb els mercats de les diferents illes.

Arribant a Ithaca

El primer pas de qualsevol aventura serà arribar fins a **Ithaca**. Mitjançant la comanda:

```
$ CONNECT ITHACA
Connected to Ithaca.
```

l'Odysseus establirà una connexió amb la IP i el port d'Ithaca especificats al seu fitxer de configuració.

En establir la connexió, Odysseus s'identificarà utilitzant el seu nom. Aquest identificador permetrà a Ithaca saber quin navegant està realitzant cada viatge i mantenir-ne l'estat encara que aquest abandoni posteriorment la connexió per fer-se a la mar.

Un cop a Ithaca, l'Odysseus podrà consultar els encàrrecs disponibles:

```
$ LIST VOYAGES
--- AVAILABLE VOYAGES ---
1. OdysseusBow      -> Ismarus   [1050]
2. TridentOfPoseidon -> Scheria   [1500]
4. CircesWand        -> Ogygia    [1000]
```

Ithaca retornarà únicament els viatges que encara estiguin disponibles, indicant-ne l'identificador, l'objecte, la destinació i la recompensa. Quan el navegant n'esculli un, podrà acceptar-lo mitjançant:

```
$ ACCEPT <id>
Voyage <id> accepted.
Destination: <destination island>.
```

Ithaca comprovarà que el viatge continuï disponible i l'assignarà a aquell Odysseus. Com que diversos navegants poden intentar acceptar encàrrecs simultàniament, caldrà garantir que **un mateix viatge mai pugui ser assignat a més d'un Odysseus**.

En aquesta fase, Ithaca enviarà tota la informació necessària sobre el viatge, però **encara no transferirà el fitxer corresponent a l'objecte**. La transferència dels objectes s'implementarà durant la Fase 3.

Un cop acceptat l'encàrrec, l'Odisseus podrà fer-se a la mar mitjançant la comanda SAIL <island>. Des d'**Ithaca**, l'única destinació vàlida serà **Aeaea**, la localització de la qual coneix gràcies al seu fitxer de configuració. A partir d'aquí, només podrà navegar cap a illes connectades directament amb el port on es trobi, utilitzant les IPs i ports que vagi descobrint durant el viatge.

```
$ SAIL Aeaea
Leaving Ithaca...
Sailing to Aeaea...
```

En sortir, l'Odisseus es desconnectarà d'Ithaca, que mantindrà l'encàrrec com a assignat mentre el navegant sigui a la mar.

Navegant entre les illes

Quan un Odisseus arribi a una illa, establirà una connexió amb el procés **Island** corresponent. Cada illa disposarà d'una capacitat màxima de port, definida al seu fitxer de configuració.

Si hi ha una plaça disponible, l'Odisseus rebrà permís per atracar. En cas contrari, la connexió es mantindrà oberta i haurà d'esperar davant de la costa fins que quedi una plaça lliure. Island gestionarà l'espera amb una cua FIFO. Quan quedi lliure una plaça, enviarà una nova trama amb DOCKED al primer Odisseus de la cua per indicar-li que ja pot entrar al port.

El procés Island haurà de continuar atenent els vaixells que ja es trobin al port mentre altres esperen. Quan un Odisseus marxi, mori o perdi la connexió, la seva plaça s'haurà d'alliberar.

Esperant davant del port

Els mars de Posidó no perdonen els vaixells mal preparats. Mentre un Odisseus està navegant entre illes no consumirà provisions, però **sí que en consumirà mentre espera davant d'un port sense poder atracar.**

Per cada **5 segons complets d'espera**, la tripulació consumirà **7 kg dels aliments transportats**. El total de provisions correspondrà en tot moment a la suma dels quilograms de menjar que encara quedin a la càrrega. La política concreta utilitzada per consumir els aliments és lliure, sempre que la càrrega i el total de provisions es mantinguin coherents.

Per exemple:

```
....  
Waiting off the coast of Aeaea...  
The harbor is full. We drop anchor and wait.  
  
>>> The crew consumes another ration.  
Provisions remaining: 40 kg  
  
>>> The crew consumes another ration.  
Provisions remaining: 33 kg  
  
>>> The crew consumes another ration.  
Provisions remaining: 26 kg  
  
>>> Aeaea has granted access to the port.  
....
```

Tan bon punt l'illa permeti l'entrada al port, deixarà de consumir provisions.

Aquest mecanisme s'haurà d'implementar sense utilitzar **esperes actives**. L'Odysseus haurà de poder esperar tant l'autorització de l'illa com el pas del temps sense consumir CPU de manera innecessària.

Si les provisions arriben a 0 abans de poder atracar, l'Odysseus es considerarà **perdut al mar**. La connexió amb l'illa es tancarà, la missió es considerarà fracassada i el procés Odysseus finalitzarà. A partir de la Fase 3, aquesta situació implicarà també la pèrdua de l'objecte transportat.

```
....  
Provisions remaining: 2 kg  
  
The crew has run out of provisions.  
Odysseus Polyphemus has been lost at sea.
```

Si el mateix Odysseus torna a executar-se, començarà novament amb els diners i provisions inicials del seu fitxer de configuració. Quan es connecti de nou a Ithaca, la missió anterior es mantindrà assignada. Només si torna a executar LIST VOYAGES, Ithaca la marcarà com a fracassada abans de retornar els viatges disponibles.

Comerciant a port

Un cop l'Odysseus hagi rebut permís per atracar, podrà comerciar amb l'illa. Mitjançant LIST MARKET, l'illa enviarà el contingut actual del seu stock.db:

```
$ LIST MARKET
--- AEAEA MARKET ---
Barley      140 kg      7 gold/kg
OliveOil    110 kg     12 gold/kg
DriedFish   90 kg     15 gold/kg
...
Map                      50 gold
```

La comanda:

```
$ BUY <product> <amount>
```

permetrà comprar una determinada quantitat d'un aliment, expressada en **quilograms**, sempre que l'illa disposi de prou stock i l'Odysseus tingui diners suficients. El cost de l'operació serà el nombre de quilograms comprats multiplicat pel preu per quilogram del producte.

Els aliments comprats s'afegiran a la càrrega del vaixell i passaran a formar part de les seves **provisions disponibles**.

De manera equivalent:

```
$ SELL <product> <amount>
```

permetrà vendre una determinada quantitat dels aliments transportats. Els quilograms venuts desapareixeran de la càrrega i l'Odysseus rebrà els diners corresponents al preu establert per aquella illa.

Per exemple:

```
$ BUY DriedFish 5
Bought 5 kg of DriedFish for 75 gold.
$ BUY Barley 10
Bought 10 kg of Barley for 70 gold.
$ SELL DriedFish 2
Sold 2 kg of DriedFish for 30 gold.
```

A més dels aliments, totes les illes vendran el seu **mapa local** per un preu fix de **50 gold**. Els mapes no formen part del stock de l'illa i no s'esgoten.

Per mantenir la mateixa sintaxi de la comanda BUY, el mapa s'adquirirà mitjançant:

```
$ BUY MAP 1
```

En aquest cas, l'única quantitat vàlida serà 1. Si l'Odyseus disposa de prou diners, se li descomptaran els 50 gold i l'illa li enviarà totes les connexions definides a la seva secció --- ROUTES ---. El mapa no s'afegirà a la càrrega del vaixell ni es podrà vendre. S'utilitzarà per incorporar les rutes bidireccionals amb SPHRAGIS al mapa conegut pel navegant. Es podrà comprar més d'una vegada el mapa de la mateixa illa i cada compra tornarà a costar 50 gold.

Si l'Odyseus queda en una situació sense sortida per manca de recursos, és a dir, no disposa de diners ni d'aliments que pugui vendre i tampoc no pot comprar les provisions o el mapa necessaris per continuar, l'usuari haurà de finalitzar el procés amb CTRL+C. L'objecte associat al viatge es considerarà perdut i la missió fracassarà. En tornar a executar-se, l'Odyseus començarà de nou amb els recursos inicials definits al seu fitxer de configuració.

Com que diversos Odyseus poden comerciar simultàniament en un mateix port, caldrà garantir que el stock **compartit es mantingui sempre coherent**.

La comanda STATUS permetrà consultar en qualsevol moment els diners disponibles, els aliments transportats i el **total de provisions**, calculat com la suma dels quilograms de tota la càrrega, així com la localització i el viatge actual de l'Odyseus.

```
$ STATUS
--- POLYPHEMUS ---
Gold: 320

Cargo:
DriedFish: 5 kg
OliveOil: 8 kg
Barley: 12 kg
Total provisions: 25 kg

Current voyage: OracleAmphora -> Ogygia
Reward: 600

Current location: Aeaea
```

Cartografiant l'Egeu

Ithaca només indicarà a l'Odisseus **l'illa de destinació** de l'encàrrec. El camí per arribar-hi l'haurà d'anar descobrint durant la travessia comprant els mapes locals de les diferents illes.

Quan un Odisseus compri un mapa mitjançant:

\$ BUY MAP 1

l'illa li enviarà les seves connexions directes, definides a la secció --- ROUTES --- del seu fitxer de configuració. Aquestes rutes bidireccionals s'incorporaran amb SPHRAGIS al mapa conegut pel navegant.

La comanda:

\$ MAP

no descobrirà noves rutes, sinó que mostrarà en qualsevol moment el **mapa construït amb tota la informació coneguda fins aleshores**.

A mesura que l'Odisseus compri mapes en noves illes, **el seu coneixement de l'Egeu s'anirà ampliant**. No caldrà calcular automàticament cap ruta fins al destí: **serà el mateix navegant qui haurà d'interpretar el mapa i decidir per on continuar**.

Per continuar el viatge s'utilitzarà SAIL <island>. **Només es podrà navegar directament cap a una illa que estigui connectada amb l'illa on es troba actualment l'Odisseus**.

Conèixer una illa o haver-la descobert anteriorment no implica poder navegar-hi directament. Per exemple, si l'Odisseus coneix Scheria però es troba a Aeolia, només podrà executar SAIL Scheria si entre les rutes conegudes d'Aeolia hi ha una connexió directa amb Scheria.

Les connexions que una Island comunica mitjançant el seu mapa seran exclusivament les rutes que hagin estat validades prèviament per la funció de SPHRAGIS. En sortir, l'Odisseus alliberarà la seva plaça al port, tancarà la connexió actual i navegarà fins al següent procés **Island** utilitzant la IP i el port descoberts.

El temps de navegació entre illes no consumirà provisions.

Arribant al destí

Quan l'Odysseus aconsegueixi arribar i atracar a l'illa indicada com a destinació del seu viatge, haurà completat la travessia.

En aquesta Fase 2, però, **encara no s'entregarà l'objecte ni es generarà el comprovant segellat**. La transferència dels fitxers, el segellat de l'entrega i el retorn posterior a Ithaca s'implementaran durant la **Fase 3**.

Per tant, en aquesta fase serà suficient arribar correctament fins al port de destinació mantenint actiu el viatge assignat.

```
user.name@montserrat:~> ./ithaca ithaca.dat voyages.dat
Ithaca initialized. 24 voyages loaded.
Waiting for Odysseus...
```

```
>>> Polyphemus connected to Ithaca.
>>> Polyphemus requested available voyages.
>>> Voyage 2 assigned to Polyphemus.
>>> Polyphemus has left Ithaca. Fair winds!
```

```
>>> Telemachus connected to Ithaca.
>>> Telemachus requested available voyages.
>>> Voyage 4 assigned to Telemachus.
>>> Telemachus has left Ithaca. Fair winds!
```

```
user.name@montserrat:~> ./island aeaea.dat stock.db
```

```
Island Aeaea initialized.
Port capacity: 1 ship.
2 sea routes loaded.
8 products available.
```

```
>>> Polyphemus approaching Aeaea.
>>> Polyphemus entered the port.
```

```
>>> Telemachus approaching Aeaea.
>>> Port full. Telemachus waiting offshore in the FIFO queue.
```

```
>>> Polyphemus bought 5 kg of DriedFish.
>>> Polyphemus bought the local map.
```

```
>>> Polyphemus has left.
>>> DOCKED sent to Telemachus; he may enter the port.
```

Figura 11. Exemple d'execució d'Ithaca i Island.

```
user.name@montserrat:~> ./odysseus polyphemus.dat
Odysseus Polyphemus is ready to sail.
```

\$ CONNECT ITHACA

Connected to Ithaca.

\$ LIST VOYAGES

--- AVAILABLE VOYAGES ---

1. OdysseusBow	-> Ismarus	[1050]
2. TridentOfPoseidon	-> Scheria	[1500]
4. CircesWand	-> Ogygia	[1000]

\$ ACCEPT 2

Voyage 2 accepted.

Destination: Scheria.

\$ SAIL Aeaea

Leaving Ithaca...

Sailing to Aeaea...

Connected to Aeaea.

Aeaea has granted access to the port.

\$ LIST MARKET

--- AEAEA MARKET ---

Barley	140 kg	7 gold/kg
OliveOil	110 kg	12 gold/kg
DriedFish	90 kg	15 gold/kg
...		
Map		50 gold

\$ BUY DriedFish 5

Bought 5 kg of DriedFish for 75 gold.

\$ BUY MAP 1

Bought the map of Aeaea for 50 gold.

New routes have been charted.

\$ STATUS

--- POLYPHEMUS ---

Gold: 125

Cargo:

Barley: 40 kg

DriedFigs: 60 kg

DriedFish: 5 kg

Total provisions: 105 kg

Current voyage: TridentOfPoseidon -> Scheria

Reward: 1500

Current location: Aeaea

```
$ MAP
[SPHRAGIS displays the known routes]

$ SAIL Aeolia
We can't sail to Aeolia from here...

$ SAIL Scheria
Leaving Aeaea...
Sailing to Scheria...

$ SAIL Aeaea
Leaving Scheria...
Sailing to Aeaea...

Connected to Aeaea.
The harbor is full. We drop anchor and wait.

>>> The crew consumes another ration.
Provisions remaining: 98 kg

>>> The crew consumes another ration.
Provisions remaining: 91 kg

>>> Aeaea has granted access to the port.
```

Figura 12. Exemples d'execució d'Odysseus.

Es demana

En aquesta segona fase es demana:

- Implementar la comunicació mitjançant sockets entre els processos Odysseus, Ithaca i Island, seguint el protocol definit a l'Annex II.
- Permetre que múltiples Odysseus interactuïn simultàniament amb Ithaca i les diferents illes.
- Implementar CONNECT ITHACA, LIST VOYAGES i ACCEPT, garantint que un mateix viatge només pugui ser assignat a un únic Odysseus.
- Implementar SAIL <island> per començar la travessia des d'Aeaea i navegar posteriorment entre les connexions conegudes.
- Gestionar la capacitat limitada dels ports amb una cua FIFO, fent esperar els vaixells quan no hi hagi places disponibles i alliberant-les quan abandonin el port o es perdi la connexió.
- Implementar el consum d'aliments durant les esperes i detectar la mort d'un Odysseus quan se n'esgotin les provisions.

- Implementar LIST MARKET, BUY i SELL, mantenint coherent el stock de cada illa davant d'operacions concurrents i reflectint-ne els canvis al fitxer stock.db.
- Implementar la compra dels mapes mitjançant BUY MAP 1. Tots els mapes tindran un cost fix de 50 gold i no disposaran d'un stock limitat.
- Incorporar les rutes obtingudes amb els mapes al conjunt de connexions conegudes per l'Odysseus.
- Implementar MAP utilitzant la llibreria SPHRAGIS per representar les rutes conegudes.
- Implementar STATUS per consultar els diners, la càrrega, el total de provisions, la localització i el viatge actual.
- Detectar quan un Odysseus arriba a l'illa de destinació del seu encàrrec.
- Gestionar correctament les desconexions, caigudes i finalitzacions dels diferents processos.

Consideracions Fase 2:

- Les consideracions de la **Fase 1 segueixen essent vàlides.**
- A partir d'aquesta fase, les comandes no només s'hauran de reconèixer sintàcticament, sinó que només es podran executar quan siguin coherents amb l'estat actual de l'Odysseus. Per exemple, no es podrà comerciar sense estar atracat en una illa ni acceptar un viatge sense estar connectat a Ithaca.
- Totes les comunicacions s'hauran d'implementar seguint el protocol definit a l'Annex II.
- Ithaca i les diferents Island hauran de poder atendre **múltiples Odysseus simultàniament**. Un client bloquejat o esperant no podrà impedir que el servidor continuï atenent la resta.
- L'assignació de viatges, les places disponibles als ports i els stocks dels mercats són **recursos compartits** i s'haurà de garantir que el seu estat sigui sempre coherent.
- SAIL <island> només serà vàlida si <island> és una connexió directa de la localització actual de l'Odysseus. El fet que una illa aparegui en alguna altra part del mapa conegut no permet navegar-hi directament.
- **No es permeten esperes actives.**
- La desconexió o caiguda d'un Odysseus no podrà deixar una plaça del port ocupada permanentment ni provocar la finalització d'Ithaca o d'una Island.
- Les compres i vendes d'aliments s'hauran de reflectir al fitxer binari stock.db. Els mapes no formaran part d'aquest fitxer ni tindran stock limitat.

- Tots els mapes tindran un preu fix de 50 gold i només es podran adquirir mitjançant BUY MAP 1.
- Els processos no es poden dissenyar assumint que s'executaran a la mateixa màquina. **Ithaca i Aeaea s'hauran d'executar obligatòriament en màquines diferents**, i la resta de processos Island s'hauran de distribuir entre les màquines disponibles.
- No es podrà utilitzar localhost, 127.0.0.1 ni cap altre mecanisme que pressuposi que dos processos remots comparteixen màquina
- La càrrega de l'Odysseus només caldrà conservar-la durant l'execució actual. La seva persistència entre execucions s'implementarà a la Fase 3.
- En finalitzar qualsevol procés mitjançant CTRL+C, s'hauran de tancar les connexions actives i alliberar correctament tots els recursos utilitzats.
- **En aquesta fase no s'ha d'implementar cap transferència de fitxers.**
- Caldrà gestionar les caigudes dels diferents processos i mantenir l'estabilitat del sistema, informant adequadament dels errors sempre que una operació no es pugui completar.
- És molt recomanable que, abans de començar a programar, es dissenyin els fluxos de comunicació, els mecanismes de concurrència i les estructures de dades compartides, **i que aquest disseny es validi amb els monitors de l'assignatura.**

Fase 3: El Viatge deixa empremta

En aquesta última fase es completarà el cicle de les missions. Fins ara, l'**Odysseus** havia après a abandonar Ithaca, navegar entre illes i arribar fins al seu destí. Ara, però, els viatges tindran una càrrega real que haurà d'arribar a bon port.

Les imatges dels objectes necessàries per a aquesta fase es proporcionaran a través de l'eStudy. Quan Ithaca assigni una missió, entregará també a l'Odysseus l'objecte que haurà de transportar, representat mitjançant una imatge. L'Odysseus haurà de rebre-la, guardar-la i conservar-la durant tot el recorregut fins arribar a l'illa de destinació.

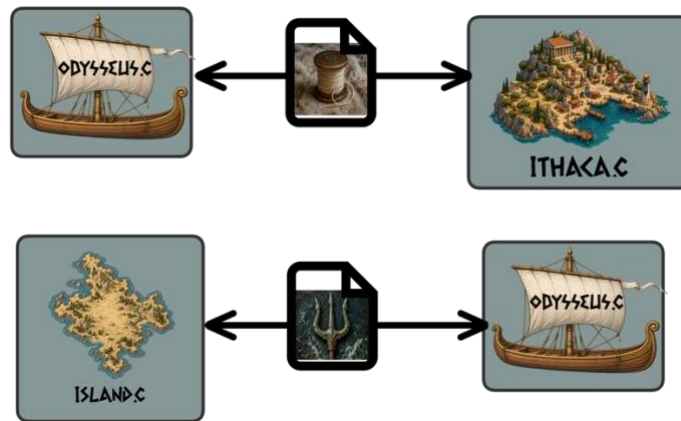


Figura 13. Esquema funcional de l'enviament d'objectes.

Un cop allà, l'objecte s'haurà d'entregar al procés **Island** corresponent. Cada illa disposa d'un glif propi que permet certificar que un objecte ha passat realment per aquell indret. Incorporar i comprovar aquests glifs es fa també amb la llibreria **SPHRAGIS**, que els processos hauran d'utilitzar quan sigui necessari.

L'illa de destinació haurà de **segellar la imatge rebuda amb el seu glif** i retornar-la a l'Odysseus. D'aquesta manera, l'objecte que havia sortit d'Ithaca es convertirà també en el comprovant del viatge: la mateixa imatge transportada, ara marcada per l'illa on s'ha fet l'entrega.

Amb la càrrega segellada de nou a bord, l'Odysseus tornarà **directament** a Ithaca executant **CONNECT ITHACA** després de tancar la connexió amb l'illa; no haurà de navegar per illes intermèdies. En reconnectar-s'hi, haurà d'entregar la imatge segellada perquè el servidor pugui comprovar que correspon a la missió que tenia assignada i que conté el glif de l'illa de destinació correcta. Només quan aquesta validació sigui satisfactòria es considerarà la missió completada i l'Odysseus rebrà la recompensa corresponent.

Així es tancarà completament el recorregut d'una missió: **Ithaca confia una càrrega a l'Odysseus, aquest la transporta fins a l'illa indicada, l'illa la segella com a prova de l'entrega i, finalment, l'Odysseus torna a casa per demostrar que ha complert la seva missió.**

Aquesta fase introduirà també la **persistència de l'estat dels processos**. El progrés ja no podrà existir únicament mentre els programes estiguin en execució: tant la informació del **stock de cada Odysseus** com l'estat del stock propi de cada **Island** s'hauran de guardar en fitxers i recuperar-se quan els processos es tornin a iniciar. D'aquesta manera, si un Odysseus o una illa abandonen el sistema i hi tornen més endavant, podran continuar a partir de l'estat assolit en execucions anteriors, en lloc de començar sempre des de zero.

```
...
$ STATUS
--- POLYPHEMUS ---
Gold: 100

Cargo:
Barley: 40 kg
Wine: 5 kg

Total provisions: 45 kg

Current voyage: TridentOfPoseidon -> Scheria
Reward: 1500
Object: trident.png
Current location: Scheria

$ DELIVER
Delivering TridentOfPoseidon to Scheria...
Object transferred correctly.

Scheria is stamping the object...

Stamped object received.

Delivery completed.
The mark of Scheria has been obtained.

$ CONNECT ITHACA
Leaving Scheria...
Returning directly to Ithaca...
Connected to Ithaca.

$ CLAIM
Presenting the stamped TridentOfPoseidon...
```

```
Object transferred correctly.
Glyph of Scheria verified.
```

```
Voyage 2 completed.
Reward received: 1500 gold.
```

\$ STATUS

```
--- POLYPHEMUS ---
Gold: 1600
```

```
Cargo:
Barley: 40 kg
Wine: 5 kg
```

```
Total provisions: 45 kg
```

```
Current voyage: None
Current location: Ithaca
```

Figura 14. Exemple d'execució d'Odisseus durant la Fase 3.

```
user.name@matagalls:~> ./island scheria.dat stock.db
Island Scheria initialized.
Port capacity: 2 ships.
2 sea routes loaded.
8 products available.

>>> Polyphemus approaching Scheria.
>>> Polyphemus entered the port.

>>> Polyphemus requests delivery of voyage 2.
>>> Receiving TridentOfPoseidon...

>>> Object received correctly.

>>> Stamping object with the glyph of Scheria...
>>> Object stamped successfully.

>>> Sending stamped object to Polyphemus...
>>> Transfer completed.

>>> Polyphemus has left Scheria.
```

Figura 15. Exemple d'execució d'una Illa durant la Fase 3.

```
user.name@montserrat:~> ./ithaca ithaca.dat voyages.dat
Ithaca initialized. 24 voyages loaded.
Waiting for Odysseus...

>>> Polyphemus connected to Ithaca.
>>> Polyphemus requested available voyages.

>>> Voyage 2 assigned to Polyphemus.
>>> Sending TridentOfPoseidon to Polyphemus...

>>> Object transferred successfully.
>>> Polyphemus has left Ithaca. Fair winds!

...

>>> Polyphemus returned to Ithaca.
>>> Polyphemus claims voyage 2.

>>> Receiving stamped TridentOfPoseidon...
>>> Object received correctly.

>>> Glyph of Scheria verified.
>>> Voyage 2 completed by Polyphemus.
>>> Reward granted: 1500 gold.
```

Figura 16. Exemple d'execució d'Ithaca durant la Fase 3.

Consideracions Fase 3:

Les consideracions de les fases anteriors segueixen essent vàlides.

- Els fitxers d'objecte s'han de transferir íntegrament i verificar amb **md5sum**.
- No és permès usar codi programat per fer l'MD5SUM. Cal executar la comanda **md5sum** que proporciona el mateix *bash* del sistema operatiu (*md5sum*).
- L'objecte transportat en una missió serà una de les imatges proporcionades a través d'eStudy. Ithaca l'enviarà a l'Odysseus, que haurà de rebre-la i emmagatzemar-la abans de començar el viatge.
- Quan l'Odysseus arribi a l'illa de destinació, haurà de transferir aquesta imatge a l'illa. El procés **Island** utilitzarà la llibreria **SPHRAGIS** per incorporar-hi el seu glif i retornarà la imatge resultant a l'Odysseus.
- L'Odysseus haurà de conservar la imatge segellada fins al seu retorn a **Ithaca**, on serà utilitzada com a comprovant de l'entrega.
- Ithaca haurà de validar que la imatge retornada correspon a la missió assignada i que conté el **glif de l'illa de destinació correcta** abans de donar la missió per completada i concedir-ne la recompensa.

- La llibreria **SPHRAGIS** serà proporcionada juntament amb la pràctica. La seva interfície i les operacions disponibles es podran consultar al fitxer de capçalera corresponent.
- Tant l'estat de l'**Odysseus** com l'stock de les diferents **Island** hauran de conservar-se entre execucions. S'haurà de guardar en un fitxer i recuperar en iniciar novament el programa, de manera que sigui possible continuar des de l'estat assolit en una execució anterior.
- No es pot utilitzar memòria secundària addicional per a emmagatzemar estat temporal de les transferències més enllà dels fitxers propis del sistema.

Requeriments de lliurament i planificació

Aquesta pràctica disposarà de diferents **punts de control o checkpoints** per poder fer-ne un seguiment acurat i garantir la consolidació de cadascuna de les fases de manera incremental. Concretament se seguirà el següent calendari de dates límit:

CONCEPTE	DEADLINE
Fase 1	04/10/2026
Fase 2	01/11/2026
Lliurament Final 1 (sobre 10)	22/11/2026
Lliurament Final 2 (sobre 10)	06/12/2026
Lliurament Final 3 (sobre 7)	10/01/2027

Els *checkpoints* no són obligatoris, tot i ser altament recomanables per poder garantir la robustesa de la pràctica i per saber que aneu en els terminis correctes per poder lliurar al final del semestre. Aquests *checkpoints* ponderen un 10% de la qualificació de la pràctica.

També és **recomanable validar el disseny global de la pràctica** amb els monitors de pràctiques abans d'iniciar la fase 1. Així podreu garantir que la implementació no patirà d'inconsistències insalvables per fases posteriors. Per això cal aprofitar els horaris de dubtes i **portar els dissenys** a validar. Per més informació consulteu Annex III. Cal recordar que una pràctica funcioni no és cap garantia que sigui vàlida, doncs pot ser que tot el disseny no compleixi els requisits de l'enunciat.

Els lliuraments es realitzaran a l'eStudy en un pou amb un fitxer que inclogui el codi font (fitxers .c, .h i el makefile) de la pràctica, funcionant completament sobre Montserrat, així com els fitxers de dades i configuració utilitzats. El fitxer haurà de ser obligatòriament en format .tar. El podeu generar amb la comanda següent:

```
tar cf Gx_Fn.tar *.c *.h makefile *.ext_fitxers
```

on Fn és el número de Fase a lliurar. Per exemple, el grup 12 enviaria G12_F1.tar per al lliurament del *checkpoint* de la Fase 1.

Per l'entrega final del Novembre en endavant, n és 3, per la Fase 3.

Qualsevol entrega que no segueixi el format d'entrega no serà corregida.

En els lliuraments finals també caldrà dipositar **memòria final en format PDF** en el tar. La memòria ha de constar, obligatòriament, dels punts que s'indiquen a continuació en l'Annex I.

Annex I: Contingut de la memòria

La memòria ha de ser una documentació externa correctament maquetada. Ha de contenir els següents apartats:

- 1. Portada**
- 2. Índex**
- 3. Disseny:** explicació de com s'ha dissenyat i estructurat la pràctica. Es pot explicar per fases o de la pràctica en global. Ha d'incloure de manera clara i entenedora:
 - a. Diagrames que expliquin quins processos s'han creat, les diferents comunicacions entre processos, etc.
 - b. Estructures de dades usades i la seva justificació.
 - c. Recursos del sistema utilitzats (*signals, sockets, semàfors, pipes*, etc.) amb la seva justificació.
Opcionals implementats.
- 4. Problemes observats i com s'han solucionat.**
- 5. Estimació temporal:** temps dedicat en el desenvolupament total de la pràctica per a cadascun dels estudiants. Les categories a tenir en compte són:
 - a. Investigació: temps dedicat a buscar eines i/o a aprendre components a implementar (fora del temps dedicat a les sessions).
 - b. Disseny: temps dedicat a dissenyar la pràctica.
 - c. Implementació: temps de codificació.
 - d. *Testing*: temps dedicat a fer proves.
 - e. Documentació: temps dedicat a escriure la memòria i a la documentació interna del codi (comentaris, etc.).
- 6. Conclusions i propostes de millora.**
- 7. (Opcional) Explicació de TOTS els noms dels processos de la pràctica. En quin tema ens hem basat aquest any per a fer l'enunciat?**
- 8. Bibliografia utilitzada** (en un format bibliogràfic correcte IEEE, APA, etc.): tant recursos bibliogràfics com enllaços.

Es valorarà la redacció i la correctesa gramatical i ortogràfica. La memòria ha de tenir **numeració de pàgines**.

La memòria s'ha d'entregar en **format PDF** juntament amb el lliurament final però es recomana anar-la elaborant durant el transcurs de la pràctica.

Annex II: Protocol de comunicació general

Per dur a terme la comunicació entre processos en diferents màquines, s'utilitzarà un protocol específic d'enviament de trames que s'explicarà a continuació. S'ha de tenir en compte que els missatges s'enviaran mitjançant *sockets* utilitzant una comunicació orientada a connexió.

En aquest protocol s'utilitzarà un **únic format de trama**, de dimensió **fixa (256 Bytes)** i formada sempre pels següents 4 camps:

Type: Camp d'1 byte que identifica el tipus de missatge enviat.

Flags: Camp d'1 byte utilitzat per indicar diferents propietats de la trama. Cada bit tindrà un significat independent, de manera que diversos flags podran estar actius simultàniament:

- **Bit 7 RESPONSE:** indica que la trama és una resposta a una petició anterior.
- **Bit 6 ERROR:** indica que l'operació sol·licitada no s'ha pogut completar correctament.
- **Bit 5 BINARY:** indica que el camp Data conté dades binàries en lloc d'informació textual.
- **Bits 4-0:** estaran reservats i s'hauran d'enviar amb valor 0.

Data Length: Camp d'1 byte en format numèric que indica el nombre de bytes útils presents al camp Data. El seu valor podrà variar entre 0 i 253.

Data: Camp de **253 bytes** que contindrà la informació associada a la trama. La interpretació d'aquest camp dependrà del valor de Type i dels flags corresponents. Quan contingui informació textual, els diferents valors es podran separar mitjançant el caràcter &, segons el format definit per a cada tipus de missatge.

TYPE (1 Byte)	FLAGS (1 Byte)	DATA LENGTH (1 Byte)	DATA (253 Bytes)
------------------	-------------------	-------------------------	---------------------

És molt important de cara a garantir la bona comunicació amb el procés que la definició de l'estructura de la trama de comunicació sigui estrictament, en ordre i format, la descrita amb anterioritat. Això vol dir que qualsevol canvi de tipus dels camps o ordre dels mateixos dins de l'estructura a dissenyar farà que les trames rebudes o enviades a procés no compleixin el protocol de comunicació i, per tant, no funcioni aquesta comunicació correctament. Si la mida del camp DATA a enviar és inferior a 253, caldrà omplir la trama amb el *padding* corresponent.

És del tot normatiu l'enviament de trames serialitzades senceres en una única escriptura al socket. És a dir, llegir amb un sol read, i escriure amb un sol write de 256B. Escriure un typedef, o struct no estarà permès. Haurà de ser una trama serialitzada.

Els bytes del camp DATA que no siguin útils s'ompliran sempre amb 0x00. DATA_LENGTH indicarà exactament quants bytes de DATA són vàlids.

Cada petició i cada resposta es defineixen com una trama separada. Les respostes conservaran el mateix TYPE de la petició, però tindran la direcció i els FLAGS corresponents.

Les combinacions habituals de FLAGS seran:

- 0x00: petició textual.
- 0x80: resposta correcta.
- 0xC0: resposta amb error (RESPONSE | ERROR).
- 0x20: dades binàries.
- 0xA0: dades binàries enviades com a resposta (RESPONSE | BINARY).

A les descripcions següents s'indiquen els valors de TYPE, FLAGS i DATA. Tot el que està entre <> s'ha de substituir pel valor corresponent; els caràcters <> no s'envien a la trama.

CONNECT ITHACA (PETICIÓ) - ODYSSEUS

Odysseus -> Ithaca

S'envia immediatament després d'establir la connexió TCP amb Ithaca.

- TYPE: 0x01
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <OdysseusName>.

CONNECT ITHACA (RESPOSTA) - ITHACA

Ithaca -> Odysseus

Ithaca confirma si ha pogut identificar l'Odysseus que ha iniciat la connexió.

- TYPE: 0x01
- FLAGS: 0x80 si la identificació és correcta; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: <Reason>.

LIST VOYAGES (PETICIÓ) - ODYSSEUS

Odysseus -> Ithaca

L'Odysseus demana a Ithaca la llista dels viatges que encara estan disponibles.

- TYPE: 0x02
- FLAGS: 0x00.
- DATA_LENGTH: 0.
- DATA: Buit.

LIST VOYAGES (RESPOSTA) - ITHACA

Ithaca -> Odysseus

Ithaca envia una trama de resposta per cada viatge disponible. Index comença a 1 i Total indica el nombre total de respostes.

- TYPE: 0x02
- FLAGS: 0x80 per a la llista; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80:
<Index>&<Total>&<VoyageID>&<Object>&<Destination>&<Reward>. Si no hi ha viatges: 0.
 - Amb FLAGS 0xC0: <Reason>.

ACCEPT (PETICIÓ) - ODYSSEUS

Odysseus -> Ithaca

L'Odysseus demana que se li assigni el viatge identificat a la trama.

- TYPE: 0x03
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <VoyageID>.

ACCEPT (RESPOSTA) - ITHACA

Ithaca -> Odysseus

A la Fase 3, després d'aquesta resposta correcta, Ithaca inicia la transferència OBJECT.

- TYPE: 0x03
- FLAGS: 0x80 si el viatge queda assignat; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80:
OK<VoyageID>&<Object>&<Destination>&<Reward>.
 - Amb FLAGS 0xC0: <Reason>.

CLAIM (PETICIÓ) - ODYSSEUS

Odysseus -> Ithaca

Després d'aquesta trama, l'Odysseus inicia la transferència CLAIM.

- TYPE: 0x04
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <VoyageID>.

CLAIM (RESPOSTA) - ITHACA

Ithaca -> Odysseus

Ithaca l'envia després de rebre i validar la imatge segellada.

- TYPE: 0x04
- FLAGS: 0x80 si la imatge és vàlida; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK<Reward>.
 - Amb FLAGS 0xC0: <Reason>.

DESCONNEXIÓ D'ITHACA (PETICIÓ) - ODYSSEUS

Odysseus -> Ithaca

S'envia abans de salpar o de tancar-se de manera controlada.

- TYPE: 0x05
- FLAGS: 0x00.
- DATA_LENGTH: 0.
- DATA: Buit.

DESCONNEXIÓ D'ITHACA (RESPOSTA) - ITHACA

Ithaca -> Odysseus

Ithaca confirma la desconnexió controlada abans que l'Odysseus tanqui el socket.

- TYPE: 0x05
- FLAGS: 0x80 si es pot tancar la connexió; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: <Reason>.

ARRIBADA A UNA ILLA (PETICIÓ) – ODYSSEUS

Odysseus -> Island

S'envia immediatament després d'establir la connexió TCP amb l'illa.

- TYPE: 0x10
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <OdysseusName>.

ARRIBADA A UNA IL·LA (RESPOSTA) - ISLAND

Island -> Odysseus

Si el port és ple, Island envia WAIT, manté el socket obert i afegeix l'Odysseus al final de la cua FIFO. Quan queda una plaça lliure, envia una nova trama 0x10 amb DOCKED al primer Odysseus de la cua.

- TYPE: 0x10
- FLAGS: 0x80 per WAIT o DOCKED; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: WAIT o DOCKED.
 - Amb FLAGS 0xC0: <Reason>.

LIST MARKET (PETICIÓ) - ODYSSEUS

Odysseus -> Island

L'Odysseus demana a l'illa el contingut actual del mercat i el preu del mapa.

- TYPE: 0x11
- FLAGS: 0x00.
- DATA_LENGTH: 0.
- DATA: Buit.

LIST MARKET (RESPOSTA) - ISLAND

Island -> Odysseus

Island envia una trama per cada producte i una darrera entrada per al mapa. Index comença a 1 i Total indica el nombre total de respostes.

- TYPE: 0x11
- FLAGS: 0x80 per a la llista; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: <Index>&<Total>&<Product>&<Amount>&<Price>.
Per al mapa: <Index>&<Total>&MAP&1&50.
 - Amb FLAGS 0xC0: <Reason>.

BUY (PETICIÓ) - ODYSSEUS

Odysseus -> Island

Aquesta trama no s'utilitza per comprar el mapa.

- TYPE: 0x12
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <Product>&<Amount>.

BUY (RESPOSTA) - ISLAND

Island -> Odysseus

Island informa si la compra s'ha completat i retorna el cost de l'operació.

- TYPE: 0x12
- FLAGS: 0x80 si la compra es completa; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK&<Product>&<Amount>&<Cost>.
 - Amb FLAGS 0xC0: <Reason>.

SELL (PETICIÓ) - ODYSSEUS

Odysseus -> Island

L'Odysseus demana vendre una quantitat concreta d'un producte que transporta.

- TYPE: 0x13
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <Product>&<Amount>.

SELL (RESPOSTA) - ISLAND

Island -> Odysseus

Island informa si la venda s'ha completat i retorna els ingressos obtinguts.

- TYPE: 0x13
- FLAGS: 0x80 si la venda es completa; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK&<Product>&<Amount>&<Income>.
 - Amb FLAGS 0xC0: <Reason>.

BUY MAP (PETICIÓ) - ODYSSEUS

Odysseus -> Island

El mapa no forma part del stock.db, no s'esgota i l'únic valor vàlid d'Amount és 1.

- TYPE: 0x14
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <Amount>.

BUY MAP (RESPOSTA) - ISLAND

Island -> Odysseus

Island processa una única compra i envia una trama per cada ruta del mapa local. Index comença a 1 i Total indica el nombre total de respostes.

- TYPE: 0x14
- FLAGS: 0x80 si la compra es completa; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK&50&<Index>&<Total>&<Name>&<IP>&<Port>. Si no hi ha rutes: OK&50&0.
 - Amb FLAGS 0xC0: <Reason>.

DELIVER (PETICIÓ) - ODYSSEUS

Odysseus -> Island

Després d'aquesta trama, l'Odysseus inicia la transferència DELIVERY.

- TYPE: 0x15
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <VoyageID>&<Object>.

DELIVER (RESPOSTA) - ISLAND

Island -> Odysseus

Island l'envia després de completar les transferències DELIVERY i SIGNED.

- TYPE: 0x15
- FLAGS: 0x80 si l'entrega i el retorn de la imatge segellada es completen; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: <Reason>.

SORTIDA D'UNA ILLA (PETICIÓ) - ODYSSEUS

Odysseus -> Island

S'envia abans de navegar cap a una altra illa o de tancar-se de manera controlada.

- TYPE: 0x16
- FLAGS: 0x00.
- DATA_LENGTH: 0.
- DATA: Buit.

SORTIDA D'UNA ILLA (RESPOSTA) - ISLAND

Island -> Odysseus

Island confirma que l'Odysseus ha abandonat el port i que la seva plaça ha quedat lliure.

- TYPE: 0x16
- FLAGS: 0x80 quan la plaça queda alliberada; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: <Reason>.

SAIL envia primer la petició de desconnexió 0x05 o 0x16, espera la resposta, tanca el socket actual, n'obre un amb la nova illa i envia la petició 0x10. Si hi ha una caiguda, el tancament del socket substitueix la desconnexió explícita.

MAP i STATUS són comandes locals i no generen trames pròpies.

Tota transferència d'imatge segueix aquest ordre: capçalera 0x20, READY 0x22, N trames de dades 0x21 i ACK final 0x23.

FileSize s'expressa en bytes ASCII i MD5SUM conté els 32 caràcters hexadecimal retornats per la comanda md5sum.

OBJECT - CAPÇALERA DE FITXER

Ithaca -> Odysseus

Ithaca inicia aquesta transferència després d'una resposta ACCEPT correcta a la Fase 3.

- TYPE: 0x20
- FLAGS: 0x80.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: OBJECT&<FileName>&<FileSize>&<MD5SUM>.

OBJECT - READY

Odysseus -> Ithaca

El receptor confirma si la capçalera és vàlida i si pot començar a rebre les dades del fitxer.

- TYPE: 0x22
- FLAGS: 0x80 si la capçalera és vàlida; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: READY.
 - Amb FLAGS 0xC0: <Reason>.

OBJECT - DADES DE FITXER

Ithaca -> Odysseus

Després de READY, l'emissor envia N trames consecutives, on N és $\text{FileSize} / 253$ arrodonit a l'enter superior.

- TYPE: 0x21
- FLAGS: 0xA0.
- DATA_LENGTH: 253 a totes les trames excepte a l'última, que conté entre 1 i 253 bytes vàlids.
- DATA: Bloc binari del fitxer.

OBJECT - ACK FINAL

Odysseus -> Ithaca

El receptor informa si ha rebut la mida esperada i si el MD5SUM del fitxer és correcte.

- TYPE: 0x23
- FLAGS: 0x80 si la mida i el MD5SUM són correctes; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: SIZE_ERROR o MD5_ERROR.

DELIVERY - CAPÇALERA DE FITXER

Odysseus -> Island

L'Odysseus inicia aquesta transferència després de la petició DELIVER.

- TYPE: 0x20
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: DELIVERY&<FileName>&<FileSize>&<MD5SUM>.

DELIVERY - READY

Island -> Odysseus

El receptor confirma si la capçalera és vàlida i si pot començar a rebre les dades del fitxer.

- TYPE: 0x22
- FLAGS: 0x80 si la capçalera és vàlida; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: READY.
 - Amb FLAGS 0xC0: <Reason>.

DELIVERY - DADES DE FITXER

Odysseus -> Island

Després de READY, l'emissor envia N trames consecutives, on N és $\text{FileSize} / 253$ arrodonit a l'enter superior.

- TYPE: 0x21
- FLAGS: 0x20.
- DATA_LENGTH: 253 a totes les trames excepte a l'última, que conté entre 1 i 253 bytes vàlids.
- DATA: Bloc binari del fitxer.

DELIVERY - ACK FINAL

Island -> Odysseus

El receptor informa si ha rebut la mida esperada i si el MD5SUM del fitxer és correcte.

- TYPE: 0x23
- FLAGS: 0x80 si la mida i el MD5SUM són correctes; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: SIZE_ERROR o MD5_ERROR.

SIGNED - CAPÇALERA DE FITXER

Island -> Odysseus

Island inicia aquesta transferència després de rebre, validar i segellar la imatge DELIVERY.

- TYPE: 0x20
- FLAGS: 0x80.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: SIGNED&<FileName>&<FileSize>&<MD5SUM>.

SIGNED - READY

Odysseus -> Island

El receptor confirma si la capçalera és vàlida i si pot començar a rebre les dades del fitxer.

- TYPE: 0x22
- FLAGS: 0x80 si la capçalera és vàlida; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: READY.
 - Amb FLAGS 0xC0: <Reason>.

SIGNED - DADES DE FITXER

Island -> Odysseus

Després de READY, l'emissor envia N trames consecutives, on N és $\text{FileSize} / 253$ arrodonit a l'enter superior.

- TYPE: 0x21
- FLAGS: 0xA0.
- DATA_LENGTH: 253 a totes les trames excepte a l'última, que conté entre 1 i 253 bytes vàlids.
- DATA: Bloc binari del fitxer.

SIGNED - ACK FINAL

Odysseus -> Island

El receptor informa si ha rebut la mida esperada i si el MD5SUM del fitxer és correcte.

- TYPE: 0x23
- FLAGS: 0x80 si la mida i el MD5SUM són correctes; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: SIZE_ERROR o MD5_ERROR.

CLAIM - CAPÇALERA DE FITXER

Odysseus -> Ithaca

L'Odysseus inicia aquesta transferència després de la petició CLAIM.

- TYPE: 0x20
- FLAGS: 0x00.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: CLAIM&<FileName>&<FileSize>&<MD5SUM>.

CLAIM - READY

Ithaca -> Odysseus

El receptor confirma si la capçalera és vàlida i si pot començar a rebre les dades del fitxer.

- TYPE: 0x22
- FLAGS: 0x80 si la capçalera és vàlida; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: READY.
 - Amb FLAGS 0xC0: <Reason>.

CLAIM - DADES DE FITXER

Odysseus -> Ithaca

Després de READY, l'emissor envia N trames consecutives, on N és FileSize / 253 arrodonit a l'enter superior.

- TYPE: 0x21
- FLAGS: 0x20.
- DATA_LENGTH: 253 a totes les trames excepte a l'última, que conté entre 1 i 253 bytes vàlids.
- DATA: Bloc binari del fitxer.

CLAIM - ACK FINAL

Ithaca -> Odysseus

El receptor informa si ha rebut la mida esperada i si el MD5SUM del fitxer és correcte.

- TYPE: 0x23
- FLAGS: 0x80 si la mida i el MD5SUM són correctes; 0xC0 en cas d'error.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA:
 - Amb FLAGS 0x80: OK.
 - Amb FLAGS 0xC0: SIZE_ERROR o MD5_ERROR.

TRAMA NACK - TOTS ELS PROCESSOS

Receptor -> emissor

S'envia quan una trama no correspon a cap format vàlid. Els errors d'una operació coneguda es responen amb el TYPE de l'operació i FLAGS 0xC0.

- TYPE: 0x7F
- FLAGS: 0xC0.
- DATA_LENGTH: Nombre de bytes vàlids de DATA.
- DATA: <Reason>, com INVALID_TYPE, INVALID_FLAGS, INVALID_LENGTH o INVALID_DATA.

Annex III: Guia per a un bon disseny d'una pràctica

Quan treballes en un sistema complex com el Nostos System, és essencial aplicar un bon disseny per gestionar la complexitat de manera eficient i escalable. El disseny ha de comptar amb conceptes clau com la modularització, la concurrència i la robustesa. Com a enginyers, és fonamental tenir clar el disseny abans de començar la implementació, ja que un plantejament ben estructurat ens permetrà detectar possibles problemes, optimitzar recursos i garantir que el projecte sigui mantenible a llarg termini. A continuació, es presenten punts generals a tenir en compte en qualsevol projecte en C, centrats en els aspectes importants d'un sistema operatiu i com es poden aplicar al context del projecte.

Modularització: Disseny de mòduls compartits

En projectes complexos com el "**Nostos System**", modularitzar correctament el codi és crucial per garantir la claredat, la reutilització i la mantenibilitat. No es tracta només de dividir el sistema en processos independents com **Odysseus i Island**, sinó també d'identificar funcionalitats comunes i encapsular-les en **mòduls compartits**. Aquests mòduls, utilitzats per diferents processos, són essencials per evitar la duplicació de codi i assegurar una major eficiència.

En qualsevol projecte de certa complexitat, és fonamental determinar quines parts del codi poden ser comunes entre els diferents processos i encapsular-les de manera independent. Si diverses funcionalitats es repeteixen en diferents punts del sistema, com la manipulació de dades o l'accés a recursos compartits, és molt més eficient organitzar-les en mòduls reutilitzables. Això facilita la mantenibilitat del sistema, ja que redueix la duplicació de codi i centralitza els canvis: qualsevol modificació en un mòdul es propagarà automàticament als processos que l'utilitzin, estalviant temps i errors potencials.

Cada **mòdul** hauria de tenir una **responsabilitat clara** i ser fàcilment reutilitzable sense modificacions per part dels processos que l'utilitzen. Això no només millora l'estructura del codi, sinó que permet **centralitzar actualitzacions** i mantenir un sistema més net, escalable i fàcil de gestionar.

Un bon disseny modular en C implica la separació del codi en **fitxers .h** i **fitxers .c**. Els **fitxers .h** contenen declaracions de funcions, constants i tipus de dades que poden ser compartits entre mòduls o processos, mentre que els **fitxers .c** implementen la lògica funcional. Aquesta separació no només facilita l'organització del projecte, sinó que també permet que el compilador gestioni millor les dependències, cosa que millora la integració i compilació del codi.

A l'hora de modularitzar, és important seguir aquests **criteris**:

1. **Reutilització:** Identifica les funcionalitats necessàries en diferents processos i encapsula-les en mòduls reutilitzables. Això evita la duplicació de codi i fa que el sistema sigui més fàcil de mantenir.

2. **Clarificació de responsabilitats:** Cada mòdul ha de tenir una responsabilitat única i clara. No barrejis funcionalitats diferents dins d'un mateix mòdul per evitar confusions i complicar la seva reutilització.
3. **Independència dels mòduls:** Els mòduls no han de dependre fortament entre ells. Si hi ha dependències, han d'estar ben definides i documentades per evitar problemes futurs d'integració o manteniment.

Finalment, és important evitar l'ús de **variables globals** dins dels mòduls compartits. Les variables globals haurien de residir únicament en els processos principals com **Odysseus.c**, mentre que els mòduls han de funcionar únicament amb funcions que rebin tots els arguments necessaris com a paràmetres. Això no només millora la modularitat del codi, sinó que també ajuda a evitar conflictes quan aquests mòduls són utilitzats per diversos processos en diferents contextos del sistema.

Concurrencia: Apropar-se al món real

En el disseny d'un sistema operatiu o d'un sistema complex com el "**Nostos System**", és fonamental entendre que, en el món real, moltes tasques han de succeir simultàniament per garantir el funcionament correcte del sistema. La **concurrència** permet que múltiples usuaris, processos o recursos treballin alhora, cosa que fa que el sistema pugui respondre a múltiples peticions de forma eficient.

Per exemple, si diversos usuaris estan fent peticions al mateix temps (com els processos **Odysseus**), la concurrència assegura que aquestes es puguin atendre simultàniament, evitant colls d'ampolla i millorant l'eficiència general del sistema. A més, és possible que un mateix procés hagi de rebre, processar i enviar respostes al mateix temps. Això exigeix un disseny que permeti gestionar diverses operacions paral·leles sense interferir entre elles.

A l'hora de dissenyar un sistema concurrent, pots fer-te preguntes per garantir que estàs prenent decisions ben fonamentades com:

1. **Com gestionaràs múltiples connexions simultànies?**
Quina és la millor estratègia per gestionar múltiples usuaris alhora? Com asseguraràs que el sistema no es bloqueja si augmenta el nombre de peticions?
2. **Com garantiràs la robustesa del sistema?**
Què passarà si un fil o procés falla mentre està atenent una petició? Com asseguraràs que el sistema pugui continuar funcionant sense interrupcions? Quines tècniques de recuperació automàtica pots implementar per mantenir la continuïtat del servei?
3. **Com controlaràs l'accés a recursos compartits**
Si diversos fils o processos han d'accedir al mateix recurs alhora, com gestionaràs aquest accés de manera segura? On creus que podrien sorgir conflictes d'accés simultani?
4. **Com optimitzaràs l'ús dels recursos del sistema?**

Estàs fent un ús òptim de la CPU i la memòria? Com pots assegurar-te que no estàs generant més processos o fils dels necessaris? Quin impacte pot tenir sobre el sistema la creació massiva de fils o processos? Com pots evitar la saturació de recursos?

5. Quin és el millor enfocament per cada part del sistema?

Cada part del sistema necessita el mateix tipus de solució concurrent o és possible que diferents parts del sistema requereixin enfocaments diferents? Què seria més òptim per les diferents tasques que s'han de realitzar?

En el disseny concurrent d'un sistema no es tracta només de garantir que funcioni correctament, sinó que funcioni **bé sota càrrega** i en condicions **reals**. La clau està en trobar solucions que equilibren **robustesa** i **eficiència**, permetent que el sistema sigui flexible i resisteixi diferents situacions. Això implica reflexionar contínuament sobre les preguntes adequades durant el disseny de cada part del sistema, per trobar l'enfocament més adequat en funció dels requisits de **rendiment** i **estabilitat**.

Com a enginyer, has de ser capaç d'analitzar **críticament** cada component del sistema i adaptar-te als requisits particulars de cada procés. Això significa buscar el millor compromís entre **rendiment**, **robustesa** i **eficiència**, tenint en compte que el que és òptim per una part del sistema pot no ser-ho per una altra. La **concurrència** és una eina poderosa, però la seva correcta aplicació depèn de comprendre a fons les necessitats del sistema. La solució perfecta en un context pot no ser aplicable a altres escenaris, i part de la teva tasca és ajustar i adaptar aquestes solucions per assegurar que cada part funcioni de manera òptima en el seu context particular.